

Report - S11L4

Lavorare con File di Testo nella CLI:SciTE, nano e File di Configurazione

Obiettivo del Lab

Il lab S11L4 introduce l'utilizzo degli editor di testo nella CLI Linux, distinguendo tra editor grafici (SciTE) e editor a riga di comando (nano). La prima parte esplora l'apertura e il salvataggio di file con SciTE, la seconda introduce nano per la navigazione e modifica da terminale. La terza parte applica queste competenze su file di configurazione reali: la personalizzazione del prompt Bash tramite PS1 in .bashrc e la modifica della configurazione nginx per cambiare porta e percorso root del server web. Il bonus approfondisce i comandi fondamentali della shell Linux.

Parte 1 - Editor di Testo Grafici (SciTE)

SciTE (SCIntilla Text Editor) e' un editor grafico leggero disponibile su Linux e Windows, usato in CyberOps per modificare file di testo e script. E' avviabile sia dalla GUI (Applications > CyberOPS > SciTE Text Editor) sia direttamente dal terminale con **scite <nomefile>**.

Step 1 - Apertura di SciTE e creazione di space.txt

Dopo aver aperto SciTE dalla GUI, viene digitato il testo della citazione di Douglas Adams e salvato tramite File > Save As con il nome **space.txt** nella home di analyst. Chiudendo e riaprendo SciTE, il file non e' visibile nell'elenco predefinito perche' l'estensione .txt non rientra tra quelle associate di default all'editor.

D: Sei riuscito a trovare subito space.txt?

R: No. Il file non viene visualizzato immediatamente nell'elenco di apertura predefinito di SciTE, perche' l'editor filtra per estensioni note. Per visualizzarlo e' necessario selezionare "All Files (*)" dal menu a discesa del filtro nella finestra di apertura.

Step 2 - Apertura di SciTE dal terminale

E' possibile aprire SciTE direttamente dal terminale passando il nome del file come argomento. In questo caso il terminale rimane bloccato finche' la finestra di SciTE e' aperta, poiche' il processo figlio occupa il processo padre (la shell).

```
[analyst@secOps ~]$ scite space.txt
```

D: Perché il prompt non viene mostrato nel terminale?

R: Il prompt non viene mostrato perché il terminale ha avviato SciTE in primo piano come processo in foreground. Finché la finestra dell'editor rimane aperta, la shell rimane occupata a gestire quel processo e non restituisce il controllo all'utente. Per riottenere il prompt senza chiudere SciTE si può avviarlo in background con **scite space.txt &**.

Parte 2 - Editor di Testo da Riga di Comando (nano)

nano è un editor di testo a riga di comando, semplice e disponibile di default nella maggior parte delle distribuzioni Linux. Non richiede interfaccia grafica ed è ideale per modificare file di configurazione da remoto o in ambienti headless.

```
[analyst@secOps ~]$ nano space.txt
```

Aprendo space.txt con nano, il testo della citazione appare su un'unica riga molto lunga, troncata a destra poiché è stata salvata senza newline. nano usa un carattere speciale per segnalare la continuazione della riga oltre i bordi dello schermo.

D: Quale carattere usa nano per rappresentare che una linea continua oltre i bordi dello schermo?

R: L'editor nano usa il carattere \$ (segno del dollaro), posizionato all'estrema destra della riga, per indicare che il testo continua oltre il limite visibile della finestra. Premendo il tasto End o la freccia destra si scorre verso la parte nascosta della riga.

Parte 3 - Lavorare con i File di Configurazione

Step 1 - Esplorazione della home e di .bashrc

Con **ls -l** si elencano i file visibili nella home di analyst. Con **ls -la** si aggiunge l'opzione per mostrare i file nascosti (quelli il cui nome inizia con un punto). Tra questi è presente **.bashrc**, il file di configurazione della shell bash che viene eseguito ad ogni apertura di un nuovo terminale interattivo.

```
[analyst@secOps ~]$ ls -l
[analyst@secOps ~]$ ls -la
[analyst@secOps ~]$ cat .bashrc
```

D: Perché i file di configurazione delle applicazioni utente sono salvati nella directory home e non sotto /etc con tutti gli altri file di configurazione a livello di sistema?

R: I file in /etc sono configurazioni di sistema globali e richiedono i privilegi di root per essere modificati. I file nella home dell'utente, invece, sono di sua proprietà esclusiva: ogni utente può personalizzare il comportamento delle proprie applicazioni senza privilegi elevati e senza interferire con la configurazione degli altri utenti o del sistema.

Step 2 - Modifica del colore del prompt Bash tramite PS1

Il file `.bashrc` contiene la variabile `PS1` che definisce l'aspetto del prompt. I codici ANSI per il colore sono: **32** = verde, **31** = rosso, **33** = giallo. Il lab prevede due modifiche successive: prima verde>rosso con SciTE, poi rosso>giallo con nano.

Codice ANSI	Colore	Nota
<code>\033[01;32m</code>	Verde (default CyberOps)	Prima modifica: da cambiare in 31
<code>\033[01;31m</code>	Rosso	Seconda modifica: da cambiare in 33
<code>\033[01;33m</code>	Giallo	Stato finale dopo entrambe le modifiche

Prima modifica (SciTE): aprire `.bashrc` con SciTE, cercare il valore 32 nella variabile `PS1` e sostituirlo con 31. Salvare e aprire un **nuovo terminale** per vedere il prompt in rosso.

D: Anche la finestra del terminale già aperta ha cambiato colore da verde a rosso? Spiega.

R: `.bashrc` viene letto dalla shell solo all'avvio di una nuova sessione interattiva. La finestra già aperta ha caricato la configurazione precedente e non si aggiorna automaticamente. Per applicare le modifiche senza aprire un nuovo terminale è necessario ricaricare la shell con il comando **bash**.

Seconda modifica (nano): aprire `.bashrc` con nano, localizzare il valore 31 in `PS1` e sostituirlo con 33. Salvare con `Ctrl+X, Y, Invio`. Ricaricare la shell con **bash** per vedere il prompt in giallo.

```
[analyst@secOps ~]$ nano .bashrc
# Modifica: \033[01;31m --> \033[01;33m
[analyst@secOps ~]$ bash
```

Step 3 - Modifica della configurazione nginx

Il file `/etc/nginx/custom_server.conf` configura un server web nginx con parametri personalizzati. Il lab richiede di modificare due valori: la porta di ascolto (da 81 a 8080) e il percorso della directory root (da `html/` a `html/text_ed_lab/`).

```
# Apertura con nano (riga -l mostra i numeri di riga):
[analyst@secOps ~]$ sudo nano -l /etc/nginx/custom_server.conf

# Riga 39: listen 81; --> listen 8080;
# Riga 47: root /usr/share/nginx/html/; --> root /usr/share/nginx/html/text_ed_lab/;
```

Dopo aver salvato, si avvia nginx con la configurazione modificata e si apre Firefox su **127.0.0.1:8080** per verificare che la pagina del lab venga servita correttamente.

```
[analyst@secOps ~]$ sudo nginx -c /etc/nginx/custom_server.conf
# Apri Firefox: http://127.0.0.1:8080
[analyst@secOps ~]$ sudo pkill nginx
```

D: A cosa si riferisce il messaggio di errore (quando si avvia nginx senza sudo)?

R: L'utente analyst non ha i permessi per aprire porte inferiori a 1024 o per scrivere nei socket di sistema di nginx. Il comando deve essere preceduto da **sudo** per essere eseguito con i privilegi di root, che sono necessari per gestire i processi di rete e accedere ai file di configurazione in /etc.

D: Appare la pagina web dopo sudo pkill nginx?

R: No. Dopo aver eseguito pkill nginx il server viene arrestato e il browser mostra un errore di connessione ("Impossibile raggiungere il sito"), perché non c'è più nessun processo in ascolto sulla porta 8080.

Domanda sfida: Puoi modificare /etc/nginx/custom_server.conf con SciTE? Descrivilo.

D: E' possibile modificare /etc/nginx/custom_server.conf con SciTE?

R: Sì, è possibile. Poiché il file si trova sotto /etc, la modifica richiede privilegi di root. È sufficiente avviare SciTE con sudo dal terminale: **sudo scite /etc/nginx/custom_server.conf**. In questo modo l'editor viene eseguito come root e permette di modificare e salvare il file di configurazione senza errori di permesso.

Bonus - Prendere Familiarità con la Shell Linux

Il bonus approfondisce i comandi fondamentali della shell Linux: navigazione del filesystem, gestione di file e directory, utilizzo delle pagine man e degli operatori di redirectione.

Parte 1 - Basi della Shell

D: Elenca alcune sezioni incluse in una pagina man.

R:

- **NAME:** nome del comando e breve descrizione
- **SYNOPSIS:** sintassi di utilizzo con opzioni e argomenti
- **DESCRIPTION:** descrizione dettagliata del comportamento
- **OPTIONS:** elenco dei flag disponibili
- **EXAMPLES:** esempi pratici di utilizzo
- **AUTHOR:** informazioni sull'autore del comando

D: Qual è la funzione del comando cp?

R: Il comando **cp** copia file nel filesystem locale. Crea una nuova copia del file sorgente nella posizione di destinazione indicata, lasciando intatto il file originale. Con l'opzione **-r** copia intere directory in modo ricorsivo.

D: Quale comando useresti per trovare maggiori informazioni sul comando pwd? Qual e' la funzione del comando pwd?

R: Per trovare informazioni su pwd si usa **man pwd**. Il comando **pwd** (print working directory) stampa a schermo il percorso assoluto della directory corrente in cui si trova l'utente.

D: Qual e' la directory corrente?

R: La directory corrente mostrata dal prompt e' **/home/analyst**, ovvero la home dell'utente analyst sulla macchina secOps.

D: In quale cartella ti trovi dopo cd cyops_folder3?

R: Ci si trova all'interno di **/home/analyst/cyops_folder3**.

D: Digita il comando 'cd ~' e descrivi cosa succede. Perche'?

R: Il comando **cd ~** riporta l'utente direttamente nella propria directory home (**/home/analyst**). Il simbolo **~** (tilde) e' una scorciatoia della shell che viene espansa automaticamente con il percorso della home dell'utente corrente.

D: Cosa succede con cd . (punto singolo)?

R: Non avviene alcun cambiamento visibile. Il punto singolo rappresenta la directory corrente stessa, quindi **cd .** istruisce la shell a spostarsi nella stessa posizione in cui si trova gia'.

D: Cosa succede con cd .. (due punti)?

R: La shell sposta la posizione corrente di un livello verso l'alto, portando l'utente nella directory genitore di quella attuale.

Prompt di partenza	Comando	Directory risultante
[analyst@secOps ~]\$	cd ..	/home
[analyst@secOps home]\$	cd ..	/
[analyst@secOps /]\$	cd ..	/ (rimane invariata: la radice non ha parent)

D: Nessun output e' stato mostrato dopo la redirectione con >. E' previsto? Spiega.

R: Si', e' previsto. L'operatore **>** redirige lo standard output del comando verso un file invece che verso lo schermo. Tutto l'output viene convogliato nel file indicato, quindi nulla viene stampato a terminale.

D: Cosa e' successo al file dopo una seconda redirectione con >?

R: Il contenuto precedente del file e' stato completamente **sovrascritto**. L'operatore **>** tronca il file a zero byte prima di scrivere il nuovo contenuto.

D: Cosa e' successo al file con l'operatore >>?

R: Il nuovo messaggio e' stato **aggiunto in fondo** al file, mantenendo intatto il contenuto precedente. L'operatore >> esegue un append senza sovrascrivere.

D: Quanti file vengono visualizzati con ls -l?

R: Con **ls -l** vengono visualizzati solo i file e le cartelle non nascosti, per un totale di 5 elementi visibili nella home di analyst.

D: Con ls -la quanti file in piu' vengono mostrati? E' possibile nascondere directory con un punto? Fornisci tre esempi di file nascosti.

R:

- Con **ls -la** vengono mostrati molti piu' elementi: tutti i file e le directory il cui nome inizia con un punto (file nascosti).
- Si', e' possibile nascondere intere directory aggiungendo un punto all'inizio del nome.
- Tre esempi di file nascosti nell'output: **.bash_history**, **.bash_profile**, **.bashrc**. Esempi di directory nascoste: **.cache**, **.config**, **.local**.

Parte 2 - Copiare, Eliminare e Spostare File

```
cp /home/analyst/some_text_file.txt /home/analyst/cyops_folder2/some_text_file.txt
```

D: Identifica i parametri nel comando cp sopra. Quali sono i file sorgente e destinazione?

R:

- **File sorgente:** /home/analyst/some_text_file.txt
- **File destinazione:** /home/analyst/cyops_folder2/some_text_file.txt

D: Quale comando hai usato per spostare il file nella home?

R: Per spostare some_text_file.txt da cyops_folder2 alla home si usa il comando: **mv cyops_folder2/some_text_file.txt /home/analyst/**

Riflessione

D: Quali sono i vantaggi dell'utilizzo della riga di comando Linux?

R:

1. **Gestione remota:** permette di operare su server e sistemi privi di GUI tramite connessioni SSH, modificando file di configurazione e gestendo processi da remoto.
2. **Controllo e flessibilita':** gli operatori di redirectione (>, >>, |) permettono di combinare comandi e manipolare flussi di dati in modo efficiente.
3. **Efficienza operativa:** la CLI richiede risorse minime rispetto a una GUI ed e' lo strumento standard nei contesti di amministrazione di sistema e cybersecurity.